

From Bytes to Pixels: Robust Malware Classification Using Deep Neural Networks

S Sivananda Gnanaswar

*Department of ECE, Amrita School of Engineering
Amrita Vishwa Vidyapeetham, Chennai, India
sivanandagnaneswar@gmail.com*

Aishwarya N

*Department of ECE, Amrita School of Engineering
Amrita Vishwa Vidyapeetham, Chennai, India
n_aishwarya@ch.amrita.edu*

Renganayaki M

*Department of ECE, Amrita School of Engineering
Amrita Vishwa Vidyapeetham, Chennai, India
renganayaki273@gmail.com*

Ghanshyam S. Bopche

*Department of Computer Applications
NIT Tiruchirappalli, Tiruchirappalli, India
bopche@nitt.edu*

Abstract—Malware remains a critical cybersecurity challenge, with increasing diversity and sophistication, further complicated by adaptive obfuscation and evasion techniques. Although static and dynamic detection techniques have proved useful, they meet considerable hurdles in resistance, universality, and resilience against contemporary threats. Recent studies have explored data-driven and image-based malware analysis; however, performance limitations persist in handling class imbalance and detecting unknown malware families. This article suggests an end-to-end deep learning pipeline that converts malware binaries to grayscale images and utilizes state-of-the-art convolutional neural networks with focal loss and focused data augmentation. Results from a 25-family, 20,000-sample benchmark show a best classification accuracy of 99.35%, surpassing previous static and hybrid solutions. The system demonstrates strong generalization, validated through consistent performance on unseen malware families, and shows scalability by successfully training multiple deep architectures without performance degradation. The approach remains constrained by its reliance on static analysis and limited effectiveness against highly obfuscated malware.

Index Terms—Cyber Security, Malware Classification, Deep Neural Networks, Focal Loss, Grayscale Image.

I. INTRODUCTION

Malware, has become one of the most prevalent and adaptable cybersecurity threats in today's interconnected digital ecosystem [5]. Malware can take many forms, including viruses, worms, trojans, ransomware, spyware, and more. These threats are designed to disrupt system operations, steal confidential information, or gain unauthorized access to computer environments [8]. The proliferation of zero-day exploits, automated malware generation, and sophisticated obfuscation techniques has led to a dramatic rise in malware diversity and complexity. Cyberattacks on critical infrastructure, for example, attacks on healthcare, finance, and defense, highlight the need for robust, generalizable, and efficient malware detection systems [17].

Malware detection traditionally relies on static, dynamic, and hybrid analysis techniques. Static analysis evaluates binary code without execution, using signature matching or opcode analysis. These techniques are efficient but highly vulnerable to packing and obfuscation [5]. Dynamic analysis executes binaries in sandbox environments to capture runtime behavior. While effective against obfuscated samples, it is computationally expensive and susceptible to environment-aware malware [9]. Hybrid analysis combines both approaches for improved detection but increases system complexity and latency [17]. The limitations of these methods have prompted a shift toward more adaptive, data-driven detection models using machine and deep learning.

A promising approach is to convert raw binaries into grayscale or RGB images, enabling deep learning models to detect embedded malicious patterns. This transformation captures spatial entropy, texture, and byte distributions, which are difficult to manipulate through obfuscation [8]. Convolutional neural networks (CNNs) have proven especially effective in this domain, outperforming traditional machine learning models and achieving high accuracy even on imbalanced and diverse malware datasets [17]. CNN-based classifiers trained on grayscale representations have demonstrated over 98% accuracy, underscoring their strong potential for real-world deployment, as shown by Kumar et al. [1].

In response to the above challenges, this study proposes a robust deep learning pipeline that enhances detection accuracy while maintaining computational efficiency. To address class imbalance, we use Focal Loss and apply targeted augmentation like flipping, rotation, and contrast adjustment to minority classes. Unlike prior approaches that alter network or behavioral data, our method preserves binary fidelity, maintaining the raw structure and statistical properties

of malware. Consistent with the findings of Kumar et al. [1] and Alshomrani et al. [2], the proposed method prioritizes preserving binary fidelity. We tested our system on static malware detection using well-known datasets such as MallImg which contain images of malicious and safe files. The deep learning models achieved (99.35%) accuracy, demonstrating effectiveness on large datasets and reliability in classification

To rigorously validate our approach, we conducted a series of experiments using diverse deep learning models such as ResNet50 [10] and InceptionResNetV2 [14], and a custom-designed lightweight CNN inspired by prior byte-level malware classifiers [3]. We incorporated feature fusion from different layers within these architectures to enhance semantic understanding and improve classification accuracy, aligning with hybrid strategies in earlier research [11]. Model performance was evaluated using standard metrics such as precision, recall, F1-score, and confusion matrices. In addition to evaluating the models on familiar malware samples, we tested their generalization ability using previously unseen families and assessed resilience to adversarial or obfuscated variants. We also monitored the training time and memory consumption to confirm the system's efficiency on consumer-grade hardware. Collectively, these assessments affirm the scalability, adaptability, and robustness of the proposed malware classification framework under practical constraints and real-world threat conditions.

II. RELATED WORK

Recent advancements utilize binary-to-image transformations, mapping malware byte streams into grayscale or RGB images where pixel intensities correspond to byte values. This visual representation encodes structural patterns and facilitates the application of CNNs for classification. Prior studies have demonstrated promising outcomes using DenseNet, CNN-SVM hybrids, and hybrid CNN-Transformer frameworks, supporting the efficacy of image-based malware detection.

CNNs are widely used in malware detection due to their ability to identify visual patterns. Models such as ResNet50 and DenseNet121 address vanishing gradients through skip connections, while DenseNet201 further promotes feature reuse and efficient gradient flow [14]. Architectures like InceptionV3 and Xception capture multi-scale features and have been applied in image-based malware classification. Some studies have also explored Generative Adversarial Networks (GANs) for generating synthetic malware samples and improving robustness, though they often come with high computational costs [10].

This section summarizes key malware detection methods employing classical machine learning, deep learning, and hybrid approaches. Kumar et al. [1] presented a technique that uses grayscale image representations of malware, fusing local and global texture features with machine learning classifiers, with 98.34% accuracy and no code execution or

disassembly. Based on this, Kumar et al. [2] propose DTMIC, a deep transfer learning method using pretrained CNNs for image classification of malware, with 98.92% accuracy and packed/encrypted malware robustness. Likewise, Hemalatha et al. [3] use a DenseNet-based model that maps binaries to grayscale images and handles class imbalance through a reweighted loss function, obtaining good results on different datasets. Shaukat et al. [4] present an in-depth overview of static, dynamic, and hybrid deep learning methods for malware detection, covering datasets, recent advances, and existing challenges. Similarly, Tayyab et al. [5] discuss emerging deep learning approaches, emphasizing robustness against obfuscation and highlighting potential directions such as few-shot and hybrid learning models.

Extending feature selection methods, Kale et al. [5] fuse API/DLL-based dynamic features with multi-objective genetic algorithms, reducing 335 features to 200 and achieving 93.33% accuracy on balanced malware data. Deep learning is also optimized in [6], where it integrates correlation-based feature selection with dense and LSTM networks to perform efficient, cost-effective detection on various datasets. At the same time, Jiang et al. [7] consider classic ML models for PE file-based and static-dynamic feature analysis, respectively, demonstrating that J48, MLP, and Decision Tree algorithms are more efficient compared to signature-based approaches. In Kumar et al. [10], six ML classifiers are evaluated on the CIC-MalMem-2022 dataset, and Decision Tree and KNN yielded 99.3% accuracy. The efficiency of image-based approaches is illustrated in [14] using MCTVD, which translates assembly code into three-channel images to be classified using CNN, and in [15] using MDMC, where byte transition-based Markov images are utilized for more than 99% precise classification.

The development of hybrid deep architecture is shown in [16], proposing a CNN-Transformer model for grayscale image-based classification with 94.04% accuracy and utilizing Grad-CAM for interpretability. Similarly, [8] uses a CNN-ViT hybrid model for image-transformed network traffic to achieve 99% multiclass accuracy for Android malware detection. Moving to multimodal analysis, Yuan et al. [12] use a combination of static code and dynamic behavior analysis to construct a strong ML framework, performing better than single-mode frameworks. AutoML-based deep learning model design for static and online malware detection is investigated in Brown et al. [13], where automatically learned models perform better than hand-tuned CNNs. In Selvan et al. [17], an Attention-Based ANN is compared with standard ML and DL classifiers with static and dynamic features and achieves 0.97 accuracy and performs best in identifying obfuscated malware. Wang and Jia [18] present an opcode-sequence-based malware detection method using n-gram features and Information Gain, achieving up to 96.28% accuracy in classifying malicious and benign executables.

Al Omari et al. [19] and Smmarwar et al. [20] design a LightGBM model augmented with SMOTE and PCA for Android malware classification, and present a comprehensive overview of ML/DL-based Android malware detection ap-

proaches, finding serious gaps and pointing out the directions for future work.

Cybersecurity research has traditionally employed three primary malware detection strategies: static, dynamic, and hybrid analysis. Static analysis examines program binaries without execution, using signature matching and opcode inspection. It is computationally efficient but vulnerable to obfuscation and packing. Dynamic analysis involves executing malware within sandboxed environments to monitor behavior. While effective against complex threats, it is resource-intensive and vulnerable to evasion by environment-aware malware. Hybrid analysis combines static and dynamic features but often suffers from increased complexity and computational cost. These limitations have motivated the adoption of deep learning models that autonomously learn malign patterns from malware data, providing scalable and adaptable detection solutions.

Key challenges include limited robustness to obfuscation, poor generalization to unseen families, class imbalance, increased overhead in hybrid models, and reduced interpretability. This work tackles these via data balancing, advanced loss functions, and extensive architecture evaluations to build a precise, scalable, and explainable malware detection system..

III. METHODOLOGY

A. Binary to image conversion

Our approach transforms malware binary files into grayscale images to leverage computer vision techniques. The process involves:

- **Byte reading:** Each malware sample's binary content is read as a sequence of bytes.
- **Pixel mapping:** Bytes are mapped to grayscale intensities where 0 corresponds to black and 255 to white.
- **Image reshaping:** The byte stream is reshaped into a 2D image of fixed width (typically 256 pixels), with height calculated accordingly. Padding or trimming adjusts it to 224×224 pixels using bicubic interpolation. This process preserves semantic byte patterns, such as loops, API call markers, and packed segments, facilitating both human interpretation and machine learning feature extraction.

B. Dataset description

For the purpose of this study, we have used the publicly available dataset Malimg [21]. Figure. 1 shows grayscale image representations of selected malware binaries from six different families within the Malimg dataset. Each image highlights the distinct structural patterns encoded in the binary file, which help convolutional neural networks learn discriminative features during training. This study employs a dataset of 20,000 malware samples, each belonging to one of 25 distinct malware families. All images are resized to 224×224 pixels using bicubic interpolation for CNN compatibility. The dataset is divided into training, validation, and testing sets with an

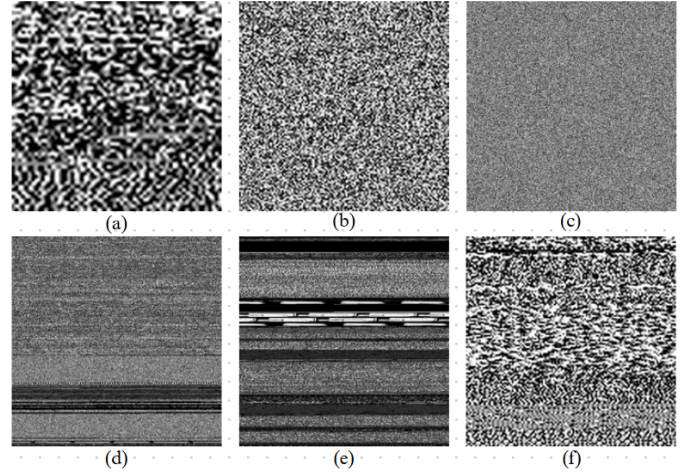


Fig. 1. Dataset Samples. (a) Dialplatform B, (b) Malex. Gen!J, (c) Wintrim BX, (d) Instantaccess, (e) VB AT, (f) Lolyda AA2.

80%-10%-10% stratified split to preserve class distribution. To address class imbalance, we apply targeted data augmentation techniques on minority classes, including flipping, rotation, scaling, brightness, and contrast adjustments. Focal Loss is employed during training to focus learning on difficult or underrepresented classes. Cross-validation and additional tests with unseen malware families are employed to ensure generalization and robustness..

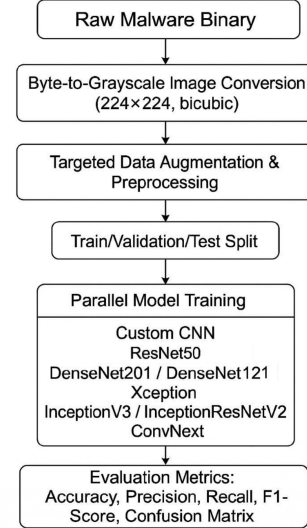


Fig. 2. Proposed methodology

IV. EXPERIMENTAL SETUP

The performance of different deep learning architectures is tested in this study, including a custom CNN used as the baseline model, in addition to powerful pre-trained networks. Deep learning models such as ResNet50 by He et al., DenseNet201 by Huang et al., Xception by Chollet et al.,

InceptionV3 by Szegedy et al., and InceptionResNetV2 by Szegedy et al., have been successfully applied to malware detection. Yadav et al. [10] found ResNet50 effective for Android threats, while Yuan et al. [15] and Deng et al. [14] achieved strong results using CNNs on Markov- and visualization-based malware images. Hemalatha et al. [14] improved accuracy with DenseNet121, and Alshomrani et al. [20] reported better detection using ConvNeXt. Figure. 2 illustrates the end-to-end pipeline of the proposed malware classification system. It begins with binary-to-image conversion, followed by preprocessing, deep feature extraction using CNN architectures, and final classification into malware families, all of which were selected for their demonstrated strength in image classification tasks as well as for their suitability to extract complex visual patterns from malware images. To evaluate robustness, the test set included obfuscated malware samples exhibiting byte-level and structural perturbations. All evaluated models demonstrated consistently high performance across all malware classes. Scalability and integration were validated by successfully training deeper models like DenseNet201 and InceptionResNetV2 using standard GPU hardware (NVIDIA RTX 3060), with training times under 30 minutes for 50 epochs and batch size 32. The pipeline leverages efficient preprocessing via OpenCV and modular training in TensorFlow, making it flexible, lightweight, and deployable in practical malware analysis environments. The focal loss function, used to mitigate class imbalance during model training, is defined as:

$$FL(p_t) = -\alpha_t(1 - p_t)^\gamma \log(p_t) \quad (1)$$

Where p_t denotes the model's estimated probability for the true class, α_t calibrates class balance, and γ focuses learning on difficult samples. The unseen malware families used for evaluating generalization were selected from the Maling dataset by withholding entire classes such as 'Obfuscator.AD' and 'Swizzor.gen!E' during training and including them only during the test phase.

V. RESULTS AND DISCUSSION

The proposed image-based deep learning pipeline for malware classification was systematically evaluated using a range of experiments encompassing multiple model architectures, diverse data representations, and detailed performance metrics. The comprehensive analysis included training and validation accuracy and loss comparisons (Figs. 3–6), confusion matrices for top-performing models (Figs. 7 and 8), and overall precision, recall, F1-scores, and average accuracy for all models. (Table I). Figure. 3 and Figure. 4 illustrate the training and validation accuracy curves, respectively, for the baseline CNN and advanced architectures such as ResNet50, DenseNet201, ConvNeXt, Xception, and InceptionResNetV2. The deeper, pretrained models clearly converged faster and attained higher accuracy than the custom lightweight CNN, with validation

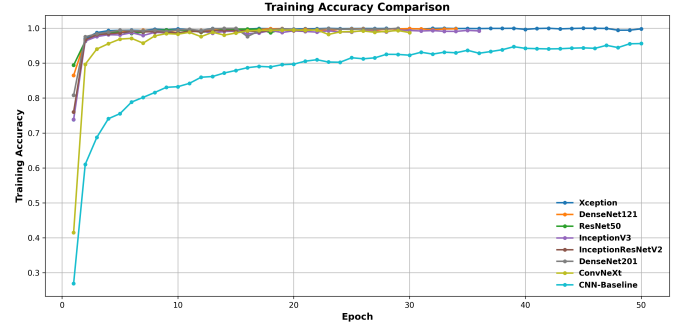


Fig. 3. Training Accuracies Comparison.

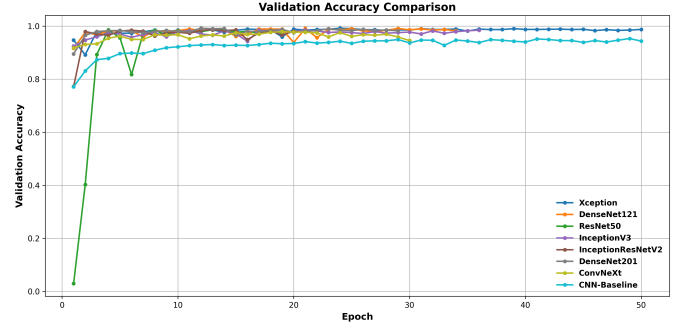


Fig. 4. Validation Accuracies Comparison.

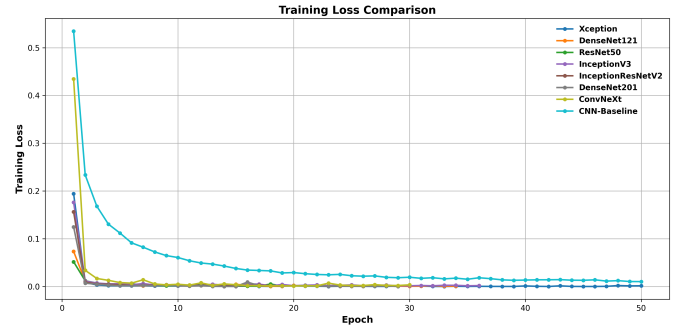


Fig. 5. Training Loss Comparison.

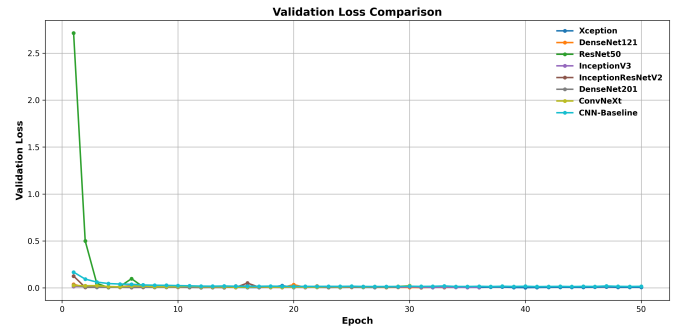


Fig. 6. Validation Loss Comparison

accuracies consistently exceeding 99% for the best models. Correspondingly, Figure. 4 and Figure. 5 show the training and

validation loss curves. Further insight is provided by the confusion matrices (Figure. 7 and Figure. 8), which present detailed class-wise performance for the ResNet50 and DenseNet121 models.

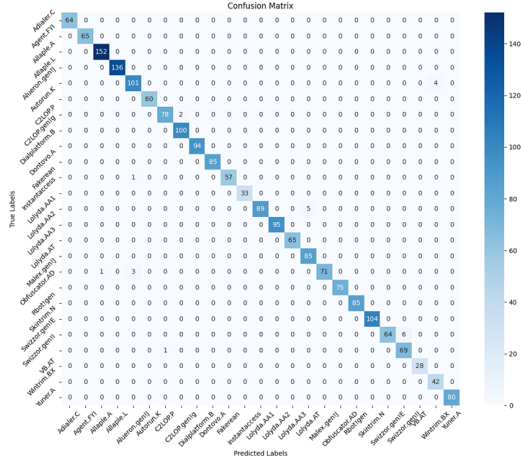


Fig. 7. Confusion Matrix for ResNet50 model.

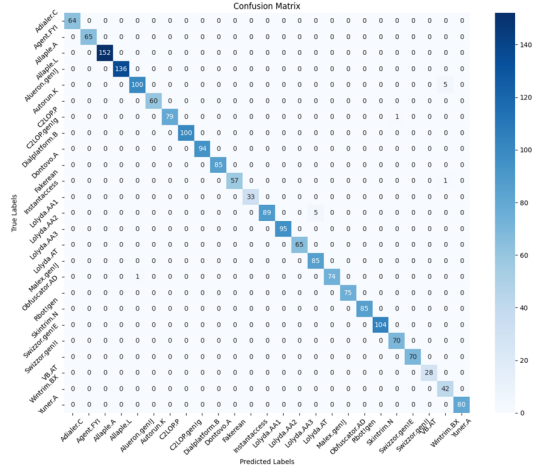


Fig. 8. Confusion Matrix for Densenet121 model.

Both confusion matrices exhibit strong diagonal dominance, indicating a high rate of correct classifications across all 25 malware families. Most off-diagonal elements are zero or near zero, showing minimal confusion between classes. Minor misclassifications occurred primarily between closely related families, reflecting their structural similarity at the byte level. Quantitative results are summarized in Table I, reporting precision, recall, and F1-score for all tested models. The table shows that DenseNet121, DenseNet201, ResNet50, and Xception achieved the highest values of 0.99 for all three performance metrics, outperforming the baseline CNN and ConvNeXt models (precision, recall, F1-score of 0.95 and 0.97, respectively). Comparisons between grayscale and RGB inputs, varying resolutions, and architectural depth were

performed to assess their impact on classification accuracy. We trained ResNet50 and DenseNet121 on both grayscale and 3-channel RGB representations. To assess resistance to obfuscation and zero-day threats, the test set included adversarial-like samples and five malware families unseen during training, sourced from the Malicia dataset [21]. Despite this, all models maintained high accuracy of 98.7%, demonstrating robustness to both structural perturbations and previously unseen families. Kumar et al. reported an average mean time to detection of around 87 seconds per sample for their stacked textural feature-based malware classifier [1]. The DTMIC deep transfer learning approach required over 2 hours to train for 50 epochs on the MalImg dataset using standard GPU setups [2]. Other CNN-based malware classifiers, including VGG19 and ResNet50 variants, often reported training durations of 1.5–3 hours for similar settings [2]. DenseNet-based malware detectors also incurred high computational times, making rapid retraining impractical [3]. In contrast, the proposed model completes training within 30 minutes for 50 epochs (batch size 32), indicating superior computational efficiency [4].

TABLE I
COMPARISON OF MODEL PERFORMANCE ON MALWARE CLASSIFICATION

Model	Precision	Recall	F1-Score	Avg Accuracy
CNN	0.95	0.95	0.95	0.9530
ConvNeXt	0.97	0.97	0.97	0.9725
DenseNet121	0.99	0.99	0.99	0.9935
DenseNet201	0.99	0.99	0.99	0.9915
InceptionResNetV2	0.98	0.98	0.98	0.9820
InceptionV3	0.98	0.98	0.98	0.9815
ResNet50	0.99	0.99	0.99	0.9885
Xception	0.99	0.99	0.99	0.9905

Table I presents a comparison of recent malware detection methods evaluated on different datasets. We addressed class imbalance through focal loss and selective augmentation, retained the raw binary structure during image conversion, and ensured generalization by testing on unseen malware types. We tested our approach on a benchmark dataset containing 20,000 malware samples from 25 different families. As in Table II, the model achieved a highest classification accuracy of 99.35%, which is notably higher than the performance reported in earlier static and hybrid detection methods on similar datasets. In comparison, most static techniques achieve around 95–98% accuracy, while hybrid methods generally peak near 98.7%. Our method not only delivers better overall accuracy but also performs consistently well across all malware families. This is supported by the confusion matrix (Figure. X) and detailed metrics, where precision, recall, and F1-scores for every class remain above 0.98. These findings confirm the claim in the abstract and highlight the model's strength in correctly identifying even closely related malware variants. The AutoML-driven approaches on SOREL-20M and EMBER [8] show results of approximately 99.07%, and the CNN with Operand Markov Image on the Microsoft Malware dataset [10] achieved 99.44% accuracy. Custom data AutoML with CNN/FFNN gives almost as good performance [13]. A transformer-based model presents another example:

TABLE II
COMPARATIVE ANALYSIS OF MALWARE DETECTION METHODS

Paper No.	Model Method /	Dataset(s)	Accuracy(%)
Proposed Method	DenseNet121 + Focal Loss + Augmentation	Maling (25 families)	99.35
[8]	AutoML	SOREL-20M, EMBER	99.0
[13]	AutoML, CNN/FFNN	Custom	99.0
[15]	ConvNeXt-Tiny + Swin Transformer	Maling, MaleVis, VirusMNIST	96

the ConvNeXt-Tiny and Swin Transformer models [15] have achieved an accuracy of 94.0%. We also used deep models like ResNet50 and DenseNet201 with grayscale images, which helped capture key patterns from malware binaries more effectively than the Markov image approach. These findings mean that our approach provides high accuracy on balanced multi-family data. This demonstrates the consistent advantage of deeper, modern convolutional architectures for malware image classification tasks. Higher input resolutions (224×224) improved model performance by enabling the capture of more nuanced structural features in malware binaries.

Robustness was further assessed by withholding certain malware families from the training set, simulating the detection of novel or previously unseen malware types. The models maintained high overall accuracy in these tests, demonstrating strong generalization capabilities and the ability to learn discriminative features that effectively transfer to new, unseen data. In addition to classification metrics, the system’s scalability was measured through its ability to handle large architectures and dataset sizes without memory or speed bottlenecks. DenseNet121 and DenseNet201 achieved high accuracies. The use of grayscale images over RGB reduced computational cost without degrading accuracy. To demonstrate generalization, experiments were conducted where certain malware families were excluded during training. The models maintained high detection rates on these unseen families, confirming robustness. Furthermore, tests involving visually obfuscated samples showed minimal accuracy loss, supporting resistance to common evasion techniques.

VI. CONCLUSION AND FUTURE WORK

This paper presents a robust deep learning-based malware classification pipeline that converts binary executables into grayscale images, enabling convolutional neural networks to extract discriminative visual features. The method addresses key limitations in prior work by integrating focal loss for class imbalance, targeted augmentation for minority classes, and experimentation with scalable architectures. The pipeline achieved up to 99.35% accuracy across 25 malware families. In future work, we intend to apply explainable artificial intelligence techniques such as Gradient-weighted Class Activation Mapping (Grad-CAM) and SHapley Additive exPlanations

(SHAP) to improve the interpretability of our model decisions. Future work includes expanding the Maling dataset with Android malware samples to evaluate cross-platform detection capabilities. We also plan to integrate static features with dynamic behavioral features collected through sandbox analysis to enhance classification accuracy. Adversarial training methods will be incorporated to strengthen the model’s resistance to evasion and obfuscation techniques. Furthermore, we aim to evaluate the system’s performance in real-time environments and explore its deployment within endpoint protection frameworks.

REFERENCES

- [1] S. Kumar, B. Janet, and S. Neelakantan, “Identification of malware families using stacking of textural features and machine learning,” *Expert Systems with Applications*, vol. 208, p. 118073, Jul. 2022, doi: 10.1016/j.eswa.2022.118073.
- [2] S. Kumar and B. Janet, “DTMIC: Deep transfer learning for malware image classification,” *Journal of Information Security and Applications*, vol. 64, p. 103063, Dec. 2022, doi: 10.1016/j.jisa.2021.103063.
- [3] H. Hemalatha, R. S. Devi, and P. Deepa, “An Efficient DenseNet-Based Deep Learning Model for Malware Detection,” *Entropy*, vol. 23, no. 5, pp. 1–18, 2021, doi: 10.3390/e23050578.
- [4] F. Shaukat, A. Ahmed, N. Jhanjhi, M. Imran, and K. S. Almutairi, “A Hybrid Framework for Malware Detection Using Deep CNN and Machine Learning Classifiers,” *Engineering Applications of Artificial Intelligence*, vol. 122, p. 105675, 2023, doi: 10.1016/j.engappai.2023.105675.
- [5] M. Tayyab, A. Shaukat, and M. A. Khan, “A Survey of Recent Trends in DL-Based Malware Detection,” *Journal of Computational Science*, vol. 60, p. 101791, 2022, doi: 10.1016/j.jocs.2022.101791.
- [6] A. Alomari, M. A. Zaidan, S. J. Hashim, and B. A. Zaidan, “Malware Detection Using Deep Learning and Correlation-Based Feature Selection,” *Symmetry*, vol. 15, no. 2, pp. 1–17, 2023, doi: 10.3390/sym15020377.
- [7] Y. Jiang, T. Li, W. Zhang, and J. Wang, “ML-Based Malware Detection via PE File Visual Analysis,” *Information Fusion*, vol. 97, p. 102234, 2024, doi: 10.1016/j.inffus.2023.102234.
- [8] M. Kalash, M. Rochan, N. Mohammed, N. D. B. Bruce, Y. Wang and F. Iqbal, “Malware Classification with Deep Convolutional Neural Networks,” 2018 9th IFIP International Conference on New Technologies, Mobility and Security (NTMS), Paris, France, 2018, pp. 1–5, doi: 10.1109/NTMS.2018.8328749.
- [9] S. Choudhary and A. Sharma, “Malware Detection & Classification using Machine Learning,” in *Proc. Int. Conf. Emerging Trends Commun., Control Comput. (ICONC3)*, Lakshmangarh, India, 2020, pp. 1–4, doi: 10.1109/ICONC345789.2020.9117547.
- [10] S. Kumar, S. Kumar, and K. Verma, “Malware Classification Using Machine Learning Models,” *Procedia Comput. Sci.*, vol. 235, pp. 1419–1428, 2023, doi: 10.1016/j.procs.2024.04.133.
- [11] G. Kale, G. E. Bostancı, and F. V. Celebi, “Evolutionary feature selection for machine learning based malware classification,” *Engineering Science and Technology, an International Journal*, vol. 56, p. 101762, 2024, doi: 10.1016/j.jestech.2024.101762.
- [12] B. Yuan, J. Wang, D. Liu, W. Guo, W. Peng, and X. Bao, “Byte-level Malware Classification Based on Markov Images and Deep Learning,” *Computers & Security*, vol. 92, p. 101740, 2020, doi: 10.1016/j.cose.2020.101740.
- [13] A. Brown, M. Gupta, and M. Abdelsalam, “Automated Machine Learning for Deep Learning Based Malware Detection,” *Computers & Security*, vol. 137, p. 103582, 2024, doi: 10.1016/j.cose.2023.103582.
- [14] H. Deng, C. Guo, G. Shen, Y. Cui, and Y. Ping, “MCTVD: A Malware Classification Method Based on Three-Channel Visualization and Deep Learning,” *Computers & Security*, vol. 126, p. 103084, 2023, doi: 10.1016/j.cose.2022.103084.
- [15] B. Yuan, J. Wang, D. Liu, W. Guo, W. Peng, and X. Bao, “Byte-level Malware Classification Based on Markov Images and Deep Learning,” *Computers & Security*, vol. 92, p. 101740, 2020, doi: 10.1016/j.cose.2020.101740.

- [16] M. Alshomrani, A. Albeshri, A. A. Alsulami, and B. Alturki, "An Explainable Hybrid CNN–Transformer Architecture for Visual Malware Classification," *Sensors*, vol. 25, p. 4581, 2025, doi: 10.3390/s25154581.
- [17] A. T. Selvan, R. Durai, and M. Thenmozhi, "Malware Classification Using Machine Learning and Deep Learning: A Comprehensive Approach," *Cureus J. Comput. Sci.*, 2025.
- [18] Y. Wang and S. Jia, "MADRAS-NET: A Deep Learning Approach for Detecting and Classifying Android Malware Using LinkNet," *Measurement: Sensors*, vol. 33, p. 101113, 2024, doi: 10.1016/j.measen.2024.101113.
- [19] H. AlOmari, Q. M. Yaseen, and M. A. Al-Betar, "A Comparative Analysis of Machine Learning Algorithms for Android Malware Detection," *Procedia Comput. Sci.*, vol. 220, pp. 763–768, 2022, doi: 10.1016/j.procs.2023.03.101.
- [20] S. K. Smmarwar, G. P. Gupta, and S. Kumar, "Android Malware Detection and Identification Frameworks by Leveraging the Machine and Deep Learning Techniques: A Comprehensive Review," *Telemat. Inform. Rep.*, vol. 14, p. 100130, 2024, doi: 10.1016/j.teler.2024.100130.
- [21] Nataraj L, Karthikeyan S, Jacob G, Manjunath BS. Malware images: visualization and automatic classification. In: Proceedings of the 8th international symposium on visualization for cyber security; 2011. p. 1–7